

Practice Material - Semantics

- * 1. For each of the following states $\sigma_1, \sigma_2, \sigma_3$, calculate: (a) the denotation of the arithmetic expression $x + (y * 3)$; (b) the denotation of the Boolean expression $x == y \ \&\& \ x - y \leq 3$; (c) And, for each $i \in \{1, 2, 3\}$, a state σ such that $\langle \text{while } x \leq y \ \&\& \ !(x = y) \text{ do } x \leftarrow x + z, \sigma_i \rangle \rightarrow^* \sigma$ when one exists.

1. $\sigma_1 = [x \mapsto 0, y \mapsto 0]$
2. $\sigma_2 = [x \mapsto -1, y \mapsto 10]$
3. $\sigma_3 = [x \mapsto -1, z \mapsto 10]$

- ** 2. Suppose $e \in \mathcal{B}$ is an arbitrary Boolean expression and $S_1, S_2, S_3 \in \mathcal{S}$ are arbitrary statements. Prove that $\langle \text{if } e \text{ then } S_1 \text{ else } S_2, \sigma \rangle \rightarrow^* \sigma'$ if, and only if, $\langle \text{if } e \text{ then } \{ \text{if } e \text{ then } S_1 \text{ else } S_3 \} \text{ else } \{ \text{if } !e \text{ then } S_2 \text{ else } S_3 \} \rangle \rightarrow^* \sigma'$ for any states $\sigma, \sigma' \in \text{State}$.

- ** 3. Let $\mathbb{P}$ be the three element set $\{+, -, \pm\}$. We define the function $\text{sign}_\sigma : \mathcal{A} \rightarrow \mathbb{P}$ for a given state $\sigma \in \text{State}$ by recursion as follows:

$$\begin{aligned} \text{sign}_\sigma(x) &= \begin{cases} + & \text{if } \sigma(x) \geq 0 \\ - & \text{otherwise} \end{cases} \\ \text{sign}_\sigma(n) &= \begin{cases} + & \text{if } n \geq 0 \\ - & \text{otherwise} \end{cases} \\ \text{sign}_\sigma(e_1 + e_2) &= \begin{cases} + & \text{if } \text{sign}_\sigma(e_1) = + \text{ and } \text{sign}_\sigma(e_2) = + \\ - & \text{if } \text{sign}_\sigma(e_1) = - \text{ and } \text{sign}_\sigma(e_2) = - \\ \pm & \text{otherwise} \end{cases} \\ \text{sign}_\sigma(e_1 - e_2) &= \begin{cases} + & \text{if } \text{sign}_\sigma(e_1) = + \text{ and } \text{sign}_\sigma(e_2) = - \\ - & \text{if } \text{sign}_\sigma(e_1) = - \text{ and } \text{sign}_\sigma(e_2) = + \\ \pm & \text{otherwise} \end{cases} \\ \text{sign}_\sigma(e_1 * e_2) &= \begin{cases} \pm & \text{if } \text{sign}_\sigma(e_1) = \pm \text{ or } \text{sign}_\sigma(e_2) = \pm \\ + & \text{if } \text{sign}_\sigma(e_1) = \text{sign}_\sigma(e_2) \\ + & \text{if } \text{sign}_\sigma(e_1) \neq \text{sign}_\sigma(e_2) \end{cases} \end{aligned}$$

Prove by structural induction over arithmetic expressions, for any arithmetic $e \in \mathcal{A}$ and state $\sigma \in \text{State}$, that:

- If $\text{sign}_\sigma(e) = +$, then $\llbracket e \rrbracket_{\mathcal{A}}(\sigma) \geq 0$;
- And, if $\text{sign}_\sigma(e) = -$, then $\llbracket e \rrbracket_{\mathcal{A}}(\sigma) < 0$

You should treat this as a single induction proof rather than proving each cases separately.

The following question uses the notation $\sigma \sim_x \sigma'$ which indicates, for some variable $x \in \text{Var}$ and for two states $\sigma, \sigma' \in \text{State}$, that $\sigma(y) = \sigma'(y)$ for all *other* variable $y \in \text{Var} \setminus \{x\}$. For example, $[x \mapsto 2, y \mapsto 3] \sim_x [x \mapsto -100, y \mapsto 3]$ but $[x \mapsto 2, y \mapsto 3] \not\sim_x [x \mapsto 2, y \mapsto 4]$.

- *** 4. Let us suppose we introduce a new language construct for Boolean expressions so that the extended grammar is given as follows:

$$B \rightarrow \text{true} \mid \text{false} \mid B \ \&\& \ B \mid B \ \parallel \ B \mid !B \mid \text{forall } x. B$$

The denotational semantics for the new construct is given by the following equation:

$$\llbracket \text{forall } x. e \rrbracket_{\mathcal{B}}(\sigma) = \begin{cases} \top & \forall \sigma' \in \text{State}. \sigma \sim_x \sigma' \Rightarrow \llbracket e \rrbracket_{\mathcal{B}}(\sigma') \\ \perp & \text{otherwise} \end{cases}$$

with all other constructs retaining their original semantics.

- (a) Evaluate the denotation of the expressions $\text{forall } x. x = 2$ and $\text{forall } x. x \leq y \parallel y \leq x$ in the state $[x \mapsto 2, y \mapsto 2]$.
- (b) Prove that if $e \in \mathcal{B}$ and $e' \in \mathcal{B}$ are semantically equivalent Boolean expressions, then $\text{forall } x. e$ and $\text{forall } x. e'$ are semantically equivalent.
- (c) Suppose that $e \in \mathcal{B}$ is a Boolean expression such that $\llbracket e \rrbracket_{\mathcal{B}}(\sigma) = \llbracket e \rrbracket_{\mathcal{B}}(\sigma')$ for any two states $\sigma \sim_x \sigma'$. Prove that $\text{forall } x. e$ is semantically equivalent to e .

- *** 5. Consider the following program P :

$$\text{while } 1 \leq x \text{ do } \{x \leftarrow x - 1; y \leftarrow y + x\}$$

By finding a suitable loop invariant, show that $\{x = n \ \&\& \ y = m\} P \{2y = 2m + n * (n - 1)\}$ for an arbitrary integers $n, m \geq 0$. Using this result, find two distinct states $\sigma, \sigma' \in \text{State}$ such that $\langle P, \sigma \rangle \rightarrow^* [y \mapsto 10]$ and likewise for $\langle P, \sigma' \rangle \rightarrow^* [y \mapsto 10]$.